Journal d'ingénierie — Robot aspirateur (CCPP)

Format de chaque entrée : date · contexte · options envisagées · choix · justification · résultat / leçon.

2026-10-08 · Vérification de l'environnement

- **Contexte** : contrôle de l'installation ROS 2 Humble, TurtleBot3 burger et outils Python avant d'écrire du code.
- **Options envisagées** : aucune (simple vérification de conformité).
- **Choix** : aucune installation ; contrôle seul.
- **Justification** : les attendus de la section 4 du brief doivent être vérifiés avant tout développement.
- **Résultat-leçon** : tout est conforme. Ubuntu 22.04.5, `ROS_DISTRO=humble`, `TURTLEBOT3_MODEL=burger`, `ROS_DOMAIN_ID=30`, `ROS_LOCALHOST_ONLY=1`, setup de Gazebo sourcé dans `.bashrc`. 7 paquets TurtleBot3/Nav2 présents. numpy 1.21.5, matplotlib 3.5.1, pytest 6.2.5. setuptools 59.6.0 (version système, non concernée par le bug `canonicalize_version`, donc non modifiée). WSLg opérationnel (`WAYLAND_DISPLAY=wayland-0`, `DISPLAY=:0`).

2026-10-08 · Métrique near_collisions

- **Contexte** : la spécification « obstacle à moins de `robot_radius` » ne peut pas être mesurée avec le LiDAR simulé. Le `range_min` du burger (`model.sdf`) vaut 0.12 m, supérieur à `robot_radius` = 0.105 m. Les mesures inférieures à `range_min` sont invalides, donc la métrique serait toujours nulle.
- **Options envisagées** : garder la définition (métrique toujours nulle) ; seuil paramétrable plus grand ; contact physique via Gazebo (plus complexe).
- **Choix** : paramètre `near_collision_distance` = 0.15 m, sur 360°, compté sur front montant.
- **Justification** : métrique non triviale et comparable entre A et B.
- **Résultat-leçon** : confronter une spécification aux caractéristiques réelles du capteur.

2026-10-08 · Dénominateur de couverture et rotation du random walker

- **Contexte** : définition du pourcentage couvert et de la rotation de la stratégie A.
- **Choix (couverture)** : le % couvert = cellules couvertes ∩ masque atteignable / masque atteignable. Le masque est l'espace libre érodé de `robot_radius` + `safety_margin`, restreint à la composante connexe du départ, et fonction partagée dans `planning.py`. Le % est donc ≤ 100 et le dénominateur est le même pour A et B.
- **Choix (rotation)** : la rotation du random walker est faite en temps (angle / vitesse angulaire, horloge simulée) et non par odométrie, pour que le nœud n'ait que `/scan` en entrée.
- **Justification** : une comparaison A/B honnête exige le même dénominateur. Un nœud à entrée unique reste simple à expliquer.
- **Résultat-leçon** : limite notée : la rotation réelle peut différer légèrement de l'angle tiré (accélérations). Sans importance pour une stratégie aléatoire.

2026-10-08 · Revue de code du jour 1

- **Contexte** : relecture du livrable du jour 1 avant commit.
- **Options envisagées** : non applicable (revue, pas de choix d'architecture).
- **Choix** : un bloquant corrigé ; trois améliorations appliquées.
- **Justification** :

    - Bloquant : passer `run_duration:=600` au launch fait lire la valeur comme un entier. Le nœud plantait (`InvalidParameterTypeException`, paramètre déclaré en double). Correction : typage explicite avec `ParameterValue(value_type=float)`, et `int` pour `seed`.
    - Carte de vérité terrain figée à la première réception de `/map`.
    - `near_collisions` et CSV écrits seulement pendant la mesure (plus de CSV vides).
    - Début de la mesure au premier déplacement supérieur à 1 cm (paramètre `start_motion_threshold`). En stratégie B, l'activation de Nav2 ne consomme ainsi pas le temps de nettoyage. A et B sont mesurés à partir du même événement.
- **Résultat-leçon** : les arguments de launch sont typés par un parseur YAML. Typer explicitement les paramètres.

2026-10-08 · Balayage : dernière bande non couverte

- **Contexte** : en pièce vide, la répartition des lignes de balayage laissait la bande du haut non couverte. Le maximum en exécution parfaite était de 98 %.
- **Options envisagées** :
    - Ajouter une ligne en haut, proposée par le relecteur. Elle crée une ligne à une seule cellule de la précédente et casse le test d'espacement.
    - Recalculer le nombre de lignes et centrer le bloc de lignes.
- **Choix** : centrer le bloc, espacement constant.
- **Justification** : garde l'espacement constant et le test d'espacement valide.
- **Résultat-leçon** : 100 % en pièce vide simulée sur grille, 31 tests verts. Limite : les bords ne sont couverts que si le rayon de nettoyage est au moins égal à la moitié de l'espacement. Le rayon demandé (0.09 m) est arrondi à 2 cellules (0.10 m) et l'espacement demandé (0.18 m) à 4 cellules (0.20 m) : la condition rayon ≥ espacement/2 est remplie grâce à ces arrondis.

2026-10-08 · Création de la carte de référence

- **Contexte** : l'étudiant ne pouvait pas piloter le robot en téléopération clavier. Il fallait une carte de la pièce pour la stratégie B et la vérité terrain.
- **Options envisagées** : téléopération clavier ; exploration par notre `random_walker` (seed 42) pendant environ 390 s simulées, avec Cartographer.
- **Choix** : `random_walker`.
- **Justification** : exploration automatique et reproductible. Premier test du nœud en simulation : 44 changements d'état, aucun blocage.
- **Résultat-leçon** :
    - Carte de 124 × 116 cellules à 0.05 m. L'hexagone est fermé et les 9 piliers sont nets.
    - Masque atteignable calculé par `planning.py` : 5564 cellules, soit 13.9 m^2.
    - Balayage boustrophédon : 112 waypoints.
    - Simulation proche du temps réel : 10 s simulées pour 10 s murales.

2026-10-08 · Incidents opérationnels

- **Contexte** : trois incidents pendant la création de la carte.
- **Options envisagées** : non applicable.
- **Choix** : aucun choix technique ; procédure corrigée.
- **Justification** : incidents sans conséquence sur les données, mais instructifs.
    - (a) `map_saver_cli` a échoué une fois sans message clair, puis réussi au second essai. La carte n'était probablement pas encore reçue dans le délai.

    - (b) Un `kill %1` a arrêté Cartographer au lieu d'un processus de test. Relancé sans conséquence, car le robot n'avait pas bougé.
    - (c) `pkill -f <motif>` a tué le shell qui l'exécutait, car le motif apparaissait dans sa propre ligne de commande.
- **Résultat-leçon** : réessayer avant de chercher plus loin. Arrêter les processus par PID, jamais par `kill %n` ni `pkill -f`.

2026-10-08 · Limite : origine de la carte

- **Contexte** : la TF `map → base_footprint` au départ vaut environ (0.037, 0.009) m, et non (0, 0). Cartographer prend son repère de suivi (IMU) comme origine.
- **Options envisagées** : non détaillées dans les sources.
- **Choix** : accepté et documenté.
- **Justification** : en stratégie A, le TF statique identité `map → odom` introduit un décalage d'environ 4 cm. Ce décalage est inférieur à une cellule (5 cm).
- **Résultat-leçon** : limite à mentionner dans `docs/RESULTATS.md`.

2026-10-08 · Bug majeur : premier run A invalide (repère odom)

- **Contexte** : run RANDOM seed 1, terminé sans plantage. Le masque atteignable ne compte que 754 cellules au lieu de 5564. Le % reste en paliers : 0 % de progression pendant plus de 5 min, alors que la distance croît.
- **Options envisagées** : non consignées.
- **Choix** :
    - Diagnostic : le plugin `diff_drive` de Gazebo publie `odom` dans le repère monde. Au spawn, `odom → base_footprint` vaut (-2.0, -0.5), vérifié en lisant `/odom`. La carte Cartographer a son origine au spawn. Le TF statique identité `map → odom` prévu par le brief décalait donc la trajectoire de 2 m, hors de la pièce.
    - Second bug, qui masquait le premier : `map_server` relisait les pixels inconnus (gris 205, p = 0.196) comme libres, car `free_thresh = 0.25`. L'extérieur de l'hexagone était donc « libre », et le tracker s'y est raccroché (cellule libre la plus proche).
    - Corrections : TF statique `map → odom` = (2.0, 0.5), calculé dans le launch à partir de la position de spawn. `free_thresh = 0.19` dans `world.yaml`.
    - Le run invalide est conservé dans `results/invalides/` pour traçabilité.
- **Justification** : le diagnostic repose sur une mesure (lecture de `/odom`), pas sur l'hypothèse du brief. Le contrôle de cohérence (nombre de cellules atteignables attendu) a révélé le bug.
- **Résultat-leçon** :
    - Une hypothèse du cahier des charges (« `map → odom` identité ») doit être vérifiée par une mesure.
    - Un contrôle de cohérence simple détecte les erreurs silencieuses.
    - Deux bugs peuvent se masquer mutuellement : corriger le premier ne suffit pas à révéler le second.

2026-10-08 · Incident : échec du spawn du robot sous WSL

- **Contexte** : à deux reprises, `spawn_entity.py` a abandonné après 30 s avec « Service /spawn_entity unavailable ». Gazebo démarrait trop lentement.
- **Options envisagées** : non applicable (incident d'exécution).
- **Choix** : contournement par spawn manuel avec les mêmes arguments (x = -2.0, y = -0.5). Action prévue : rendre le launch robuste, avec un délai ou un timeout de spawn plus long.

- **Justification** : le tracker démarre la mesure au premier déplacement. Le temps du CSV reste donc correct malgré le spawn tardif.
- **Résultat-leçon** : sous WSL, les temps de démarrage sont variables. Prévoir des marges dans les launch files.

2026-10-08 · Premier résultat de la stratégie A (MVP jour 1)

- **Contexte** : run RANDOM seed 1, 600 s simulées. Vérifications de démarrage OK : masque de 5564 cellules, `map → odom` = (2.0, 0.5).
- **Options envisagées** : non applicable (mesure).
- **Choix** : conservation du run comme premier résultat de référence.
- **Justification** : premier CSV et première courbe du MVP, produits avec le protocole de la section 6.
- **Résultat-leçon** :
  - Couverture finale : 58.1 %. 80 % non atteint. Distance : 72.0 m (0.12 m/s en moyenne). near_collisions : 27.
  - Couverture à 60 / 120 / 180 / 300 / 450 / 600 s : 10.6 / 14.1 / 19.6 / 25.2 / 36.5 / 58.1 %.
  - Croissance irrégulière, avec des plateaux : le robot repasse sur des zones déjà couvertes. La couverture accélère en 2e moitié, quand il atteint de nouvelles zones.
  - Simulation environ 1.4 fois plus lente que le temps réel. Sans conséquence, car tout est en temps simulé.
  - Fichiers : `results/coverage_vs_time.png`, `results/summary.md`, rosbag de 24 Mo.
  - Limite : un seul run, donc aucune conclusion statistique pour l'instant.

2026-10-08 · Décision : simulation accélérée pour le protocole de la stratégie A

- **Contexte** : un run de 600 s simulées prenait environ 14 min murales (simulation à environ 0.7 × le temps réel sous WSL, fenêtres ouvertes). Le protocole demande plusieurs runs.
- **Options envisagées** : rester en temps réel avec fenêtres ; mode headless seul ; headless avec monde accéléré.
- **Choix** : launch avec deux arguments.
  - `headless:=true` : ni gzclient ni RViz.
  - `fast:=true` : copie du monde `turtlebot3_world` dans `aspi_bot/worlds/`, avec `real_time_update_rate = 0`. gzserver tourne en lockstep, pour que la physique attende la mise à jour des capteurs.
  - Spawn rendu robuste : timeout de 120 s au lieu de 30 s.
- **Justification** : méthode de validation avec des critères fixés avant le run. Même seed (1) en accéléré et run de référence en temps réel. Une courbe identique n'est pas attendue : une marche aléatoire diverge à la moindre différence de timing. On juge donc la fidélité du contrôle en temps simulé.
  - C1 : fréquence de `/scan` à 5 Hz ± 10 %.
  - C2 : fréquence de `/cmd_vel` à 10 Hz ± 10 %.
  - C3 : vitesse moyenne ± 15 %.
  - C4 : near_collisions du même ordre (rapport < 2).
  - C5 : % final ± 15 points.
- **Résultat-leçon** :
  - Résultats : C1 5.00 Hz OK ; C2 9.99 Hz OK ; C3 0.120 contre 0.12 m/s OK ; C4 24 contre 27 OK ; C5 54.4 % contre 58.1 % (écart de 3.7 points) OK.
  - Les deux courbes sont quasi superposées jusqu'à environ 5 min (écart inférieur à 1 point à 60, 180 et 300 s), puis divergent (30.1 contre 36.5 % à 450 s).

    - Facteur d'accélération de ×2.84 : 600 s simulées en environ 3.5 min murales, contre environ 14 min.
    - Décision : tous les runs officiels de la stratégie A sont en mode accéléré (le seed 1 officiel est le run accéléré). Le run temps réel est conservé dans `results/comparaison_vitesse/` comme référence de validation.
    - Vérifié en revue : `/clock` est publié à 10 Hz en temps simulé (pas de 0.1 s simulée, quelle que soit l'accélération), donc la granularité des timers ne change pas. Le lockstep ne synchronise pas les nœuds ROS : les fréquences sont donc vérifiées en temps simulé.
    - La graine du launch est aussi transmise à Gazebo (`gzserver.launch.py` déclare un argument `seed`). Choix assumé, pour une meilleure reproductibilité.
    - Limites : le facteur d'accélération dépend de la charge de la machine, à noter pour chaque run. Ne pas appliquer ce mode à Nav2 (stratégie B) sans une vérification croisée identique : la boucle de contrôle de Nav2 est probablement cadencée en temps mural. Elle serait alors ralentie en temps simulé, ce qui pénaliserait B.
    - Leçon : valider un changement de banc de test par des critères chiffrés fixés à l'avance.

2026-10-08 · Incidents mineurs : /clock et boucle d'attente

- **Contexte** : deux incidents pendant la mise au point du mode accéléré.
    - (a) Un abonné à `/clock` en QoS par défaut ne recevait aucun message.
    - (b) Une boucle d'attente laissée en arrière-plan par un run précédent a dû être arrêtée.
- **Options envisagées** : non applicable (incidents d'exécution).
- **Choix** :
    - (a) `/clock` est publié en BEST_EFFORT. L'abonnement utilise donc `qos_profile_sensor_data`.
    - (b) Aucune boucle d'attente ne doit rester en arrière-plan après un run.
- **Justification** : (a) une QoS incompatible entre éditeur et abonné empêche toute réception, sans message d'erreur explicite. (b) une boucle restante consomme des ressources et peut perturber le run suivant.
- **Résultat-leçon** : vérifier la QoS de chaque topic avant de s'abonner. Ne jamais laisser de boucle d'attente derrière soi.

2026-10-08 · Lancement du jour 2

- **Contexte** : début du jour 2, stratégie B planifiée (balayage boustrophédon exécuté par Nav2). Le travail est découpé en 10 tickets, J2-01 à J2-10, dans `docs/TASKS.md`.
- **Options envisagées** : config Nav2 issue du `burger.yaml` racine du paquet installé ; config issue de `turtlebot3_navigation2/param/humble/burger.yaml`. Pour Nav2 : simulation en temps réel ou en mode accéléré.
- **Choix** : base `turtlebot3_navigation2/param/humble/burger.yaml`. Nav2 testé d'abord en temps réel ; mode accéléré seulement après vérification croisée.
- **Justification** : le `burger.yaml` racine du paquet installé contient des éléments Jazzy (opennav_docking), inutilisables en Humble. La boucle de contrôle de Nav2 est probablement cadencée en temps mural : une accélération la pénaliserait, ce qui reste à vérifier.
- **Résultat-leçon** : écarts prévus, à documenter dans la config. `xy_goal_tolerance` passe de 0.25 à 0.15 m (espacement de balayage de 0.18 m). `stop_on_failure` est déjà à false et `robot_model_type` déjà à DifferentialMotionModel dans cette version : aucun changement nécessaire sur ces deux points.

2026-10-08 · Premier run B (réglage T0) : résultat décevant

- **Contexte** : premier run complet de la stratégie B (balayage boustrophédon exécuté par Nav2), en temps réel, 600 s.
  - Mise en route : Nav2 actif en environ 12 s ; planner et tracker d'accord (5564 cellules, 112 waypoints) ; pose de départ d'environ (0, 0).
  - Non-régression de la stratégie A après refactoring du launch : 10.5 % à 60 s (référence 10.46 %), OK.
  - Résultat B : 33.2 % couvert (A : 51.0 ± 6.7 %), 40.9 m parcourus (A : 70.8 m), 13 near_collisions. Balayage arrêté au waypoint 44/112, avec 1 waypoint manqué (indice 41).
  - Paliers : couverture bloquée entre 17 et 22 % de 3 à 8 min. Certains waypoints prennent 50 à 75 s (n° 20, 30, 32, 44), contre 3 à 20 s pour les autres.
  - Log Nav2 : 22 « Failed to make progress », 28 « Controller patience exceeded », recoveries (spin, backup), « Control loop missed its desired rate 10 Hz » répété. Facteur temps réel 0.91.
- **Options envisagées** :
  - Diagnostic (chef de projet, à confirmer par les runs de réglage) :
    - (1) La config TurtleBot3 règle `waypoint_follower.loop_rate` à 2000 Hz (défaut Nav2 : 20). Résultat : 37 000 lignes « Processing waypoint » dans le log, CPU saturé.
    - (2) Les waypoints bloquants sont tous près d'un pilier ou d'un mur. La costmap locale gonfle les obstacles de 1.0 m (globale : 0.55 m), alors que le planner place volontairement les lignes à au moins 0.155 m des obstacles. Le contrôleur DWB n'avance plus et déclenche les recoveries.
  - Corriger les deux causes d'un coup, ou un seul facteur à la fois avec mesure après chaque correction.
- **Choix** : un facteur à la fois, runs de réglage de 300 s pour gagner du temps, protocole final à 600 s.
  - T1 : `loop_rate` à 20 seul.
  - T2 : T1 + inflation à 0.30 m et `cost_scaling` à 10 dans les deux costmaps.
  - Run T0 conservé dans `results/reglages_B/`.
- **Justification** : mesurer chaque correction séparément permet d'attribuer l'effet de chacune.
- **Résultat-leçon** : une config « officielle » peut contenir des valeurs inadaptées. Les paramètres de sécurité de Nav2 (inflation) sont pensés pour la navigation point à point, pas pour la couverture, qui longe volontairement les obstacles.

2026-10-08 · Mesure de la fréquence de contrôle de Nav2

- **Contexte** : `/cmd_vel` est publié à 21 Hz en temps simulé. Cette fréquence n'est pas celle du contrôleur (10 Hz configuré).
- **Options envisagées** : non applicable (mesure).
- **Choix** : pour mesurer le contrôleur, enregistrer `/cmd_vel_nav`.
- **Justification** : `nav2_bringup` remappe la sortie du `controller_server` sur `/cmd_vel_nav` et ajoute un `velocity_smoother` (20 Hz) qui publie `/cmd_vel`. Le 21 Hz mesuré sur `/cmd_vel` vient donc du smoother.
- **Résultat-leçon** : lire le launch de bringup avant d'interpréter un topic.

2026-10-08 · Incidents d'exploitation sur la stratégie B

- **Contexte** : trois incidents lors des essais de la stratégie B.
  - (a) À l'arrêt, `component_container_isolated` (conteneur Nav2) survit à SIGINT et SIGTERM.
  - (b) À l'arrêt, `coverage_executor` levait une exception.

    - (c) L'argument de launch `params_file` fuyait vers `gzserver`.
- **Options envisagées** : non applicable (incidents d'exécution).
- **Choix** :
    - (a) arrêt par SIGKILL sur le PID du conteneur ;
    - (b) exception corrigée ;
    - (c) argument renommé `nav2_params`.
- **Justification** : (c) les arguments de launch ne sont pas cloisonnés entre nœuds inclus en ROS 2 Humble.
- **Résultat-leçon** : les arguments de launch sont partagés entre les nœuds inclus : leur donner des noms spécifiques. Un conteneur Nav2 peut survivre à l'arrêt normal : vérifier son PID après chaque run.

2026-10-08 · Stratégie B : réglages T1 et T2 (surcharge CPU et inflation)

- **Contexte** : essais de 300 s en temps réel sur la stratégie B. Pourcentage couvert à 180 s / 300 s : T0 17.4 / 19.5 ; T1 17.5 / 18.2 ; T2 21.9 / 21.9.
- **Options envisagées** : un facteur à la fois, chacun mesuré séparément. T1 : `loop_rate` à 20. T2 : T1 plus inflation à 0.30 m et `cost_scaling` à 10 dans les deux costmaps.
- **Choix** : T1 et T2 testés successivement, sans les cumuler sans mesure.
- **Justification** : T1 supprime la surcharge CPU (« Processing waypoint » : 18 741 → 215 lignes ; contrôleur à 10.15 Hz en temps simulé, facteur temps réel 0.90) mais ne change pas le comportement. T2 accélère le début (+4 points à 180 s) mais le robot reste bloqué de 177 s à la fin.
- **Résultat-leçon** : cause localisée du blocage : une ligne de balayage coupée par un pilier (waypoints (3.52, -0.67) puis (2.77, -0.67) ; pilier en x 2.96–3.27, y -0.72 à -0.42). Le robot finit le premier segment face au pilier à 0.25 m. DWB oscille (vx ≈ 0, wz ±0.03 à 0.18 rad/s) sans contourner. La surcharge CPU était un vrai défaut, mais pas la cause du blocage. Mesurer chaque correction séparément évite de conclure trop vite. Pendant T1, la TF map→odom vue par le conteneur Nav2 est restée figée à ≈ 234 s simulées (739 messages « Transform data too old » sur tout le log) alors que le flux était publié ; phénomène non reproduit en T0 ni T2, puis revu en mode accéléré et au run officiel B3.

2026-10-08 · Stratégie B : contrôleur Regulated Pure Pursuit (T3)

- **Contexte** : le contrôleur DWB reste bloqué au pilier (entrée précédente). Essai d'un autre contrôleur de Nav2, Regulated Pure Pursuit (RPP).
- **Options envisagées** :
    - changer de contrôleur ;
    - calculer les contournements dans notre planner (T4) ;
    - décomposition cellulaire de Choset (trop long pour 3 jours).
- **Choix** : changer de contrôleur (T3). Réglages alignés sur la stratégie A pour une comparaison équitable : vitesse de 0.15 m/s (celle de `random_walker` ; DWB autorisait 0.22 m/s), rotation sur place à 1.0 rad/s (celle de `random_walker`), pas de ralentissement près des obstacles.
- **Justification** : le changement de contrôleur (DWB → RPP, avec une vitesse ramenée de 0.22 à 0.15 m/s) a supprimé le blocage ; DWB n'a pas été testé à 0.15 m/s, on ne peut donc pas attribuer le gain au seul algorithme. RPP n'a pas été comparé à d'autres contrôleurs.
- **Résultat-leçon** : à 300 s, T3 atteint 28.5 % de couverture (T2 : 21.9 %), 30.5 m parcourus, 1 near_collision (T2 : 5), 0 « Failed to make progress » (T2 : 13), 0 recovery. Dernier waypoint : 37/112 démarré (36 atteints) (T2 : 19). Le passage du pilier prend environ 13 s : rotation sur place puis contournement par le sud. Limites restantes : la vitesse (0.15 m/s) et les rotations sur place (environ 48 s sur 300 s).

2026-10-08 · Stratégie B : contournements calculés par le planner (T4, impasse)

- **Contexte** : après T3, les lignes de balayage restent coupées par les obstacles. Essai de contournements calculés par notre planner.
- **Options envisagées** : garder T3 seul ; ajouter des points de contournement calculés par le planner (T4).
- **Choix** : T3 retenu. `plan_transitions` est conservé, testé, et désactivé par défaut (argument de launch).
- **Justification** : principe de `plan_transitions` : fonction pure, Dijkstra sur le 8-voisinage du masque atteignable, simplification par visibilité, tronçons ≤ 0.3 m. Résultat : 112 → 245 waypoints, 49 transitions corrigées, 0 point hors masque, 8 tests unitaires (39 tests au total). Bug trouvé pendant le développement : des tronçons à 45° passaient exactement sur un coin de cellule, et l'arrondi les rangeait hors masque. Correction par un test exact de segment. Résultat T4 (T3 plus contournements) à 300 s : 22.4 % contre 28.5 % pour T3 ; 25 à 26 waypoints d'origine atteints contre 36 ; 43 « collision ahead » (RPP), 8 « patience exceeded », 3 recoveries. Les points intermédiaires consomment 38 % du temps, car chaque but déclenche une replanification et une rotation.
- **Résultat-leçon** : avec RPP, Nav2 contourne déjà les obstacles. Cause non démontrée : notre marge (0.155 m) est au-dessus du rayon inscrit (0.10 m), donc hors zone létale ; une hypothèse est que les plans NavFn entre points intermédiaires rasent les piliers à 0.10–0.15 m. Une amélioration algorithmique « évidente » doit être mesurée : elle peut être contre-productive selon le reste de la chaîne.

2026-10-08 · Bug : fuite mémoire de coverage_executor

- **Contexte** : un executor orphelin occupait 4.8 Go de RAM (80 %) et survivait à SIGINT.
- **Options envisagées** : non consignées.
- **Choix** : boucle `spin_once` avec test du future. Sortie si rclpy n'est plus ok ou si le serveur d'action disparaît. Annulation avec délai. Paramètre `max_duration`, réglé par le launch à `run_duration` + 10 s.
- **Justification** : cause, par lecture du code de rclpy et de `nav2_simple_commander` (Humble) : le gestionnaire de signal de rclpy éteint le contexte sans lever `KeyboardInterrupt`. La boucle `while not isTaskComplete()` ne testait pas `rclpy.ok()` et tournait à vide. Chaque appel à `spin_until_future_complete` ajoutait un callback au même future, jamais résolu.
- **Résultat-leçon** : vérifié pendant T4, mémoire stable (74.7 → 74.9 Mo en 4 min). Une boucle d'attente doit toujours tester l'état du contexte ROS et avoir une sortie explicite.

2026-10-08 · Écart de 245 / 247 waypoints entre le nœud et le calcul hors ligne

- **Contexte** : le nœud produit 245 waypoints, le calcul hors ligne 247 pour la même carte.
- **Options envisagées** : non applicable (explication d'un écart constaté).
- **Choix** : aucune correction ; les deux résultats sont valides (0 transition non résolue, 0 tronçon hors masque).
- **Justification** : la résolution de `nav_msgs/OccupancyGrid` est un float32 (0.05 devient 0.05000000074…). Cet arrondi minime déplace quelques points de simplification situés exactement sur des frontières de cellules.
- **Résultat-leçon** : connaître les types des messages ROS (float32) quand on compare un calcul hors ligne et un calcul en ligne.

2026-10-08 · Stratégie B : vérification croisée temps réel / accéléré (négative)

- **Contexte** : même vérification que pour la stratégie A, appliquée à la stratégie B (config T3, seed 1, 600 s). Le risque avait été identifié en revue : les boucles de Nav2 sont peut-être cadencées en temps

mural.
- **Options envisagées** : accepter le mode accéléré pour B ; mesurer B en temps réel uniquement. Critères fixés avant les runs :
  - C1 : fréquence de `/cmd_vel_nav` à 10 Hz ± 10 % en temps simulé ;
  - C2 : fréquence de `/scan` à 5 Hz ± 10 % ;
  - C3 : % final ± 5 points ;
  - C4 : nombre de waypoints atteints ± 10 % ;
  - C5 : nombre d'incidents ≤ valeur en temps réel + 3.
- **Choix** : mesure de B en temps réel uniquement. La stratégie A reste en mode accéléré, validé par ses propres critères.
- **Justification** : ce qui est comparé est le % couvert en fonction du temps simulé. Chaque stratégie doit donc être mesurée dans un mode où son comportement en temps simulé est vérifié fidèle.
- **Résultats** (valeurs F = accéléré, R = temps réel) :
  - C1 KO : F 4.42 Hz, R 9.56 Hz ;
  - C2 OK : F 5.00 Hz ;
  - C3 « OK » en apparence : F 56.8 %, R 54.1 %. Trompeur : les courbes divergent (F 53.6 %, R 29.0 % à 300 s) ;
  - C4 KO : F 8 waypoints, R 69 ;
  - C5 KO : F 85 incidents, R 1.
  - En F, la TF `map → odom` vue par Nav2 reste figée à 89 s simulées (2 292 messages « Transform data too old »), avec 55 near_collisions. F a parcouru 510 m en zigzags et recoveries, contre 61.9 m en R. Facteur d'accélération obtenu : environ 1.9 seulement.
- **Résultat-leçon** : le mode accéléré est négatif pour B. Un critère isolé (C3) peut sembler bon par hasard : il faut plusieurs critères, dont des critères de comportement. Le risque identifié en revue était fondé.

2026-10-08 · Stratégie B : run de référence en temps réel (T3, seed 1)

- **Contexte** : premier run de référence de la stratégie B, config T3, seed 1, 600 s, en temps réel.
- **Options envisagées** : non applicable (mesure).
- **Choix** : ce run est retenu comme référence de B, en temps réel, suite à la vérification croisée négative de l'entrée précédente.
- **Justification** : seul mode de mesure validé pour la stratégie B.
- **Résultat-leçon** :
  - Couverture finale : 54.1 % (A : 51.0 ± 6.7 %). 80 % non atteint. Distance : 61.9 m. near_collisions : 1.
  - Waypoints : 69 atteints sur 112. Un « Failed to make progress », 0 recovery. Facteur temps réel : 0.95 (mesuré 0.952).
  - L'executor s'arrête seul à `max_duration` (610 s). Sa mémoire reste stable, environ 74 Mo : le correctif de fuite est confirmé.
  - Limite : un seul run, donc aucune conclusion statistique.

2026-10-08 · Incident : Nav2 bloqué à l'attente d'activation (run B)

- **Contexte** : au premier lancement du run R, l'executor est resté bloqué sur l'attente de Nav2 (`waitUntilNav2Active`). Avertissement précédent dans le log : « amcl: failed to send response to /amcl/get_state (timeout) ». Fréquence : 1 lancement sur 3.
- **Options envisagées** : non consignées.

- **Choix** : procédure. Si Nav2 n'est pas actif 90 s après le lancement, arrêter et relancer. Le cas est noté comme incident.
- **Justification** : `BasicNavigator` n'a pas de délai d'attente. La sortie doit donc être gérée à l'extérieur du nœud.
- **Résultat-leçon** : un blocage intermittent ne doit pas être pris pour un résultat. Limite à noter dans `docs/RESULTATS.md`.

2026-10-08 · Essai T5 : suppression du ralentissement aux waypoints

- **Contexte** : en revue, le relecteur a relevé deux pertes de temps dans la configuration T3. Le contrôleur RPP ralentit avant chaque but (`approach_velocity_scaling_dist`, défaut 0.6 m, jusqu'à `min_approach_linear_velocity`). Le `waypoint_follower` marque en outre une pause de 200 ms à chaque waypoint. Estimation non mesurée : plusieurs minutes perdues sur 10.
- **Options envisagées** : garder T3 ; réduire le ralentissement et la pause (T5).
- **Choix** : `approach_velocity_scaling_dist` = 0.2 m (juste au-dessus de `xy_goal_tolerance` = 0.15 m) et `waypoint_pause_duration` = 0. T5 devient la configuration officielle (`nav2_params.yaml`). Fin des réglages de la stratégie B.
- **Justification** : mesure en temps réel, 600 s, T3 (seed 1) contre T5 (seed 5).
  - Couverture à 300 s : 29.0 % contre 30.6 %.
  - Couverture à 450 s : 39.8 % contre 47.3 %.
  - Couverture finale : 54.1 % contre 59.7 %.
  - Waypoints atteints : 69 contre 73.
  - Distance : 61.9 m contre 69.8 m.
  - Vitesse moyenne en translation : 0.131 m/s contre 0.147 m/s.
  - Rotation sur place : 18 % du temps dans les deux cas.
  - Recoveries : 0 dans les deux cas. T5 a eu 2 « controller patience exceeded », soit environ 5 s perdues.
- **Résultat-leçon** : gain net sur le même protocole, mais un seul run par configuration : la différence ne suffit pas à conclure statistiquement. Restent 18 % du temps en rotation sur place et quelques arrêts d'environ 30 s sur les petits segments entre lignes ; ce sont les perspectives de la stratégie B.

2026-10-08 · Correctif : blocage de l'executor au démarrage

- **Contexte** : sur 5 lancements, l'executor est resté bloqué 2 fois au démarrage. Avertissement dans le log : « amcl: failed to send response to /amcl/get_state (timeout) ».
- **Options envisagées** : non consignées.
- **Choix** :
  - fonction locale `wait_nav2_active` : interrogation du service `GetState` avec un délai de 2 s et un nouvel essai ;
  - la pose initiale est republiée jusqu'à réception sur `/amcl_pose` ;
  - plus aucune attente sans délai dans l'executor ;
  - `rclpy` est initialisé avec `SignalHandlerOptions.NO`, pour que SIGINT lève `KeyboardInterrupt`.
- **Justification** : mécanisme identifié dans le code de `nav2_simple_commander` (Humble, attente sans délai) ; la perte de la réponse elle-même n'a pas été observée directement. `_waitForNodeToActivate` appelle `spin_until_future_complete` sans délai sur la réponse du service `GetState`. Si la réponse est perdue, le processus reste bloqué dans cette attente en code C : l'interpréteur Python ne reprend jamais la main et SIGINT n'est jamais traité. Voir aussi l'entrée « SIGINT ignoré par les processus lancés en arrière-plan » : le Ctrl+C ignoré s'explique au moins en partie par la méthode de lancement.

- **Résultat-leçon** : à valider pendant les runs officiels de la stratégie B. Une bibliothèque « simple » peut cacher une attente infinie : toute attente réseau doit avoir un délai.

2026-10-08 · Correctif : course au démarrage de Nav2 (but FollowWaypoints rejeté)

- **Contexte** : le correctif d'attente de Nav2 (`wait_nav2_active`) n'attendait que `amcl` et `bt_navigator`. Le `lifecycle_manager` active les nœuds en séquence. Le but `FollowWaypoints` partait 0.5 ms après « Nav2 actif », alors que `waypoint_follower` n'était activé que 0.1 s plus tard. Résultat : « request was rejected », 2 lancements sur 2, aucun CSV produit.
- **Options envisagées** : non consignées.
- **Choix** : attendre aussi `waypoint_follower`, puis le serveur d'action `FollowWaypoints`. Cinq essais si le but est rejeté.
- **Justification** : explication vérifiée dans le code de Nav2 en revue. Le serveur d'action existe dès la configuration du nœud, mais il rejette les buts tant que le nœud n'est pas activé. L'ancien `waitUntilNav2Active` masquait la course par des pauses de 2 s.
- **Résultat-leçon** :
  - 3 démarrages sur 3 conformes (Nav2 actif en 10 à 17 s).
  - Un correctif peut créer un autre bug. Le test d'intégration avec vérifications de démarrage l'a attrapé avant de fausser une mesure.

2026-10-08 · Protocole officiel B (config T5, temps réel, 600 s, 3 runs)

- **Contexte** : protocole officiel de la stratégie B, config T5, en temps réel, 600 s, 3 runs.
- **Options envisagées** : non applicable (mesure).
- **Choix** : résultats rapportés tels quels, sans retirer de run.
- **Justification** : comparaison A/B sur le protocole défini (3 runs, 600 s). Limite à noter : la stratégie A est mesurée en mode accéléré, la stratégie B en temps réel.
- **Résultat-leçon** :
  - B1 : 59.8 % de couverture, 70.2 m, 0 near_collision, 71/112 waypoints.
  - B2 : 55.4 %, 62.2 m, 2 near_collisions, 70/112 waypoints.
  - B3 : 17.2 %, 29.5 m, 25 near_collisions, 14/112 waypoints.
  - Moyenne B : 44.1 ± 23.4 %. A (3 runs accélérés) : 51.0 ± 6.7 %.
  - 80 % jamais atteint. Moyennes de A et B à moins de 4 points jusqu'à 540 s (B3 tire la moyenne de B vers le bas) ; B1 et B2 devant la moyenne de A dès ≈ 4 min.
  - Dispersion de B nettement plus forte que celle de A. Avec 3 runs, aucune conclusion statistique solide.

2026-10-08 · Run B3 : TF figée dans Nav2 et choix de la moyenne officielle

- **Contexte** : à environ 135 s simulées, la TF `map → odom` vue par le conteneur Nav2 reste figée (4 329 messages « Transform data too old »). Le rosbag montre qu'AMCL la publie sans interruption (5 Hz, aucun trou supérieur à 1 s). Nav2 croit le robot immobile : 42 « Failed to make progress », puis recoveries, et 71 « Behavior Tree tick rate exceeded ». Même phénomène déjà vu en essai T1 et en mode accéléré. Cause non identifiée (charge CPU sous WSL, DDS, buffer TF ?).
- **Options envisagées** : garder B3 dans la moyenne ; garder B3 et ajouter un 4e run ; diagnostiquer d'abord ; exclure B3.
- **Choix** (de l'étudiant) : garder B3 dans la moyenne officielle, et indiquer à côté « 2 runs sur 3 terminés normalement : 57.6 % ».
- **Justification** : ne pas écarter un résultat après l'avoir vu. C'est une limite de fiabilité de la stratégie B dans notre environnement, à présenter comme telle.

- **Résultat-leçon** : la fiabilité fait partie de la performance. Une stratégie plus « intelligente » dépend d'une pile logicielle plus lourde et plus fragile.

2026-10-08 · Refactoring de lisibilité de coverage_executor et non-régression

- **Contexte** : après le protocole officiel, la fonction d'attente de Nav2 de `coverage_executor` (environ 55 lignes, 3 boucles imbriquées) est difficile à défendre devant le jury. Le code a changé après les mesures.
- **Options envisagées** : non précisées dans le compte rendu source (à compléter si nécessaire).
- **Choix** : découpage de la fonction d'attente en petites fonctions (attente d'un nœud actif, attente de la pose initiale, attente du serveur d'action). Annulation protégée regroupée dans une seule fonction.
- **Justification** : un nœud = une responsabilité, et chaque fonction doit pouvoir être expliquée en une phrase. Comme le code a changé après les mesures, un essai de non-régression en simulation a été fait (120 s, temps réel).
- **Résultat-leçon** : séquence de démarrage conforme (amcl actif, pose initiale, bt_navigator actif, waypoint_follower actif, Nav2 actif, 112 waypoints). Couverture à 60 s et 120 s : 7.6 % et 15.4 %, à comparer aux runs officiels B1/B2 (8.3–8.4 % à 60 s, 15.5–16.2 % à 120 s). Arrêt automatique à `max_duration` avec annulation du but (« Goal was canceled. Stopping the robot. »). Arrêt manuel par SIGINT en 1 s. Toute modification faite après une mesure demande une non-régression, même si elle semble purement cosmétique. Il s'agit d'un seul essai, pas d'une campagne.

2026-10-08 · Découverte : SIGINT ignoré par les processus lancés en arrière-plan

- **Contexte** : pendant la journée, plusieurs processus `coverage_executor` survivaient à Ctrl+C. Un `kill -INT` envoyé directement à l'executor était ignoré. `/proc/<pid>/status` montrait SIGINT dans les signaux ignorés.
- **Options envisagées** : non consignées.
- **Choix** : relancer le test avec SIGINT remis par défaut, et ne pas modifier le code de l'executor pour ce motif.
- **Justification** : un processus lancé en arrière-plan (`&`, `nohup`) par un shell non interactif hérite d'un SIGINT ignoré. Python n'installe alors pas son gestionnaire de SIGINT. Ce mécanisme explique au moins en partie les executors survivants observés. Ce n'était donc pas uniquement un défaut de notre code.
- **Résultat-leçon** : avant de conclure à un bug du code testé, vérifier l'outil de test (signaux hérités, environnement du shell). La méthode de test a été corrigée.

2026-10-08 · Jour 3 du plan — code gelé, pas de vidéo de démonstration

- **Contexte** : le jour 3 est consacré à l'évaluation et aux livrables. Le cahier de soutenance demande « vidéo ou captures de la démo ».
- **Options envisagées** :
  - vidéo d'environ 30 s (Gazebo + RViz), enregistrée par l'étudiant sous Windows ;
  - captures : figures de `results/` (couverture en fonction du temps, courbes par run), capture RViz éventuelle.
- **Choix** (décision de l'étudiant) : pas de vidéo. Les figures de `results/` (couverture en fonction du temps, courbes par run) servent de captures de démo.
- **Justification** : choix de l'étudiant.
- **Résultat-leçon** : code gelé. Le jour 3 ne contient que l'audit de cohérence des documents, le plan des slides avec la préparation des questions du jury, et un README d'utilisation.

2026-10-08 · Préparation du robot réel (bonus), sans robot

- **Contexte** : séance prévue à l'école avec un TurtleBot3 Burger réel. Le robot n'est pas disponible pour l'instant : la préparation est testée uniquement en simulation.
- **Options envisagées** :
  - zone de test : murs en carton (plan principal) ; barrière virtuelle en repère odom (en option) ; keepout filter de Nav2 pour la stratégie B (en option) ;
  - mesure de la stratégie A : localisation AMCL (utilisée pour la mesure seulement) ou TF statique `map → odom` en repli.
- **Choix** :
  - deux launch sans Gazebo : `aspi_real_random` et `aspi_real_planned`, avec `use_sim_time` à false par défaut (true pour les tester avec Gazebo) ;
  - vitesses réduites : 0.10 m/s en translation, 0.6 rad/s en rotation ;
  - `near_collision_distance` = 0.20 m ;
  - mesure de A via AMCL, TF statique en repli ; zone limitée par des murs en carton ;
  - CSV du robot dans `results/robot_reel/` ;
  - procédure réseau WSL2 et checklist dans `docs/JOUR_ROBOT.md`. Aucune modification de `.bashrc` ni de `.wslconfig`.
- **Justification** : le LDS-02 a une portée minimale d'environ 0.16 m. Avec 0.15 m, la métrique serait toujours nulle, même piège qu'au jour 1. Le seuil de 0.20 m est au-dessus de cette portée minimale.
- **Résultat-leçon** : résultats des tests en simulation :
  - barrière A : le robot reste dans le rectangle ; arrêt à environ 0.25 m avant le bord, dépassement maximal d'environ 0.13 m ;
  - stratégie B à 0.10 m/s : 12.2 % de couverture en 120 s ;
  - keepout : fonctionne, mais un départ sur le bord de la zone autorisée empêche tout mouvement (collision détectée dès le premier pas, 75 messages « patience exceeded ») ;
  - avec une marge de 0.5 m autour du départ : 30.7 % en 120 s, et le robot reste dans la zone. Le script de masque refuse désormais une zone sans marge de 0.5 m autour du départ.

2026-10-08 · Sécurité : un Ctrl+C n'arrêtait pas le robot

- **Contexte** : en test avec Gazebo, après Ctrl+C du launch, le `random_walker` s'arrêtait sans publier de vitesse nulle. La dernière consigne (0.10 m/s) restait appliquée et le robot continuait tout droit. Le TurtleBot3 réel garde aussi la dernière consigne reçue.
- **Options envisagées** : correctif dans le nœud ; procédure seule (Ctrl+C puis arrêt manuel). La procédure a été conservée en complément du correctif.
- **Choix** : rclpy initialisé sans gestionnaire de signal ; 5 vitesses nulles publiées à l'arrêt ; SIGTERM traité comme Ctrl+C. Procédure : Ctrl+C puis toujours `stop_robot.sh`.
- **Justification** : le gestionnaire de signal de rclpy ferme le contexte avant le bloc d'arrêt, donc la publication de la vitesse nulle était sautée. Le même mécanisme avait déjà causé un bug dans l'executor.
- **Résultat-leçon** : vérifié avec Gazebo : vitesse nulle 70 ms après Ctrl+C, robot arrêté en moins de 1 s ; idem avec SIGTERM. En cours suite à la revue de code : second Ctrl+C ignoré pendant l'arrêt ; arrêt si `/scan` (ou `/odom`) n'arrive plus depuis 0.5 s ; arrêt propre du tracker. Leçon : la sécurité d'un robot réel se teste, elle ne se suppose pas.

2026-10-08 · Limites connues pour le robot réel

- **Contexte** : limites identifiées pendant la préparation de la séance, à présenter au jury et à respecter pendant les essais.
- **Options envisagées** : non applicable (constat).
- **Choix** : limites documentées ; procédure `stop_robot.sh` systématique après Ctrl+C.
- **Justification** : ces limites viennent de l'architecture (lien réseau, capteur, odométrie) et ne se corrigent pas par le code seul.
- **Résultat-leçon** :
  - si le lien PC–robot tombe, rien côté PC ne peut arrêter le robot ;
  - les obstacles sous la portée minimale du LiDAR sont invisibles ;
  - la barrière virtuelle (odom) glisse avec la dérive d'odométrie ;
  - avec la stratégie B, une dernière consigne de Nav2 peut rester appliquée après Ctrl+C, d'où l'arrêt systématique par `stop_robot.sh`.

2026-10-09 · Séance robot réel : configuration constatée

- **Contexte** : première séance avec le TurtleBot3 Burger réel. Configuration relevée pendant la séance.
- **Options envisagées** : non consignées.
- **Choix** : aucun choix technique ; configuration consignée pour les essais suivants.
  - Robot n°1, joint en point d'accès Wi-Fi à l'adresse 10.42.0.1, utilisateur SSH `turtle`, clé SSH du PC installée sur le robot.
  - `ROS_DOMAIN_ID=1`, ROS 2 Humble, LiDAR LDS-02.
  - PC sous WSL en réseau `mirrored`, pare-feu Hyper-V ouvert en entrée.
  - Carte `zone_reelle` de 11 h 11.
- **Justification** : ces valeurs diffèrent de la simulation (domaine 30, pas de SSH). Les noter évite de reproduire les incidents 1 et 2.
- **Résultat-leçon** : le robot n'a pas d'Internet et pas d'horloge sauvegardée. Il repart au 2 octobre à chaque extinction (voir incident 3).

2026-10-09 · Incident 1 : variables de la simulation dans un onglet

- **Contexte** : un onglet de terminal avait gardé `ROS_DOMAIN_ID` et `ROS_LOCALHOST_ONLY` de la simulation. La téléopération n'avait aucun effet sur le robot.
- **Options envisagées** : non consignées.
- **Choix** : non consigné (la séance ne détaille pas de correction).
- **Justification** : le domaine et le mode localhost de la simulation ne correspondent pas à ceux du robot (domaine 1). Les nœuds d'un onglet ne voient donc pas le robot.
- **Résultat-leçon** : vérifier `ROS_DOMAIN_ID` et `ROS_LOCALHOST_ONLY` dans chaque onglet avant un essai réel.

2026-10-09 · Incident 2 : deux bringups simultanés sur le robot

- **Contexte** : deux bringups ont été lancés en même temps sur le robot. Seul `/scan` était visible, sans `/cmd_vel`.
- **Options envisagées** : non consignées.
- **Choix** : non consigné (la séance ne détaille pas de correction).
- **Justification** : deux bringups concurrents se gênent sur le même robot, et la commande ne passe plus.

- **Résultat-leçon** : un seul bringup actif à la fois. Vérifier les topics attendus (`/cmd_vel`, `/odom`, `/scan`) avant chaque essai.

2026-10-09 · Incident 3 : horloge du robot au 2 octobre

- **Contexte** : sans Internet et sans horloge sauvegardée, le robot était à la date du 2 octobre, soit un écart de 7 jours exactement. Les TF étaient rejetées et la mesure figée (run RANDOM_2_20261009-113503, bloqué à 37,5 %).
- **Options envisagées** : non consignées.
- **Choix** : remise à l'heure depuis le PC. Le script `robot_check.sh` mesure l'écart et remet l'heure à jour si l'écart dépasse 300 ms.
- **Justification** : sans Internet, aucune synchronisation automatique. Une horloge décalée rend les transformations inutilisables.
- **Résultat-leçon** : mesurer l'écart d'horloge avant chaque run, car le robot repart au 2 octobre à chaque extinction.

2026-10-09 · Incident 4 : première zone trop étroite

- **Contexte** : la première carte ne couvrait que 0,95 m² (cartons trop serrés). Le run d'essai RANDOM_1_20261009-102808 a donné 79,4 % en 300 s, avec 255 near_collisions.
- **Options envisagées** : non consignées.
- **Choix** : run conservé comme essai, à exclure des résultats. Nouvelle carte `zone_reelle` de 634 cellules, soit environ 1,6 m².
- **Justification** : avec des cartons trop serrés, le LiDAR détecte un obstacle à moins de 0,20 m en permanence, d'où le nombre élevé de near_collisions.
- **Résultat-leçon** : une zone trop petite fausse les métriques. Contrôler le nombre de cellules atteignables avant chaque série de runs.

2026-10-09 · Incident 5 : dernière consigne conservée après Ctrl+C

- **Contexte** : le `random_walker` ne s'arrêtait pas en fin de run. Après Ctrl+C, le TurtleBot3 garde le dernier `/cmd_vel` reçu, et le robot a percuté un mur.
- **Options envisagées** : non consignées.
- **Choix** : paramètre `run_duration` dans le walker. Après la durée, un Twist nul est republié à chaque tick. Le script `stop_robot.sh` publie le Twist nul sur le robot via SSH, puis depuis le PC, avec ses propres exports.
- **Justification** : le robot réel garde la dernière consigne, comme la simulation (voir l'entrée du 2026-10-08 sur la sécurité).
- **Résultat-leçon** : test isolé du walker (domaine 77) : 0,1 m/s pendant le run, 0 après `run_duration`. Confirmé sur le robot par le message « fin du run : robot à l'arrêt ». Corrigé.

2026-10-09 · Incident 6 : batterie trop basse, moteurs coupés

- **Contexte** : à 11,03 V (29 %), les moteurs ont été coupés alors que le LiDAR restait actif. Le robot semblait tourner sans bouger (run seed 2, vers 11 h 19).
- **Options envisagées** : non consignées.
- **Choix** : non consigné (la séance ne détaille pas de correction).
- **Justification** : le LiDAR publie `/scan` normalement, donc le capteur ne signale pas la panne des moteurs.

- **Résultat-leçon** : contrôler la batterie avant chaque séance (plus de 11,3 V, seuil noté pour la prochaine séance). Un run où le robot ne bouge pas est invalide, même si `/scan` circule.

2026-10-09 · Incident 7 : `/tf_static` perdu sur le Wi-Fi

- **Contexte** : `/tf_static` n'arrivait pas au PC de façon fiable sur le Wi-Fi. AMCL affichait « queue is full », sa position était figée et la trace rose de RViz s'était arrêtée.
- **Options envisagées** : non consignées.
- **Choix** : publication du `/tf_static` sur le PC, avec `robot_state_publisher` du burger (script `pc_robot_description.sh`, URDF avec `${namespace}` retiré). Corrigé.
- **Justification** : un message statique publié une seule fois et perdu sur le Wi-Fi n'est pas republié. Le PC fournit le modèle du robot sans dépendre du lien.
- **Résultat-leçon** : sur un lien instable, ne pas compter sur un topic statique publié une seule fois par le robot. Vérifier l'arbre TF avant un essai de navigation.

2026-10-09 · Incident 8 : heure du PC prise trop tôt

- **Contexte** : lors de la remise à l'heure, l'heure du PC était lue avant la saisie du mot de passe sudo. Le robot était donc en retard de plusieurs secondes, et le script redemandait le mot de passe à chaque fois.
- **Options envisagées** : non consignées.
- **Choix** : corrigé. Le correctif n'est pas détaillé dans les notes de séance ; le défaut venait de la lecture de l'heure avant la saisie.
- **Justification** : l'heure envoyée doit correspondre au moment de l'envoi, pas au début du script.
- **Résultat-leçon** : prendre l'heure au moment de l'envoi de la commande, pas avant une étape interactive.

2026-10-09 · Incident 9 : couche « Coverage map » figée dans RViz

- **Contexte** : dans RViz, la couche « Coverage map » ne se rafraîchit pas toujours, alors que `/coverage_status` et le CSV continuent de progresser.
- **Options envisagées** : piste à vérifier, la durabilité Volatile dans `aspi.rviz`.
- **Choix** : non résolu.
- **Justification** : le CSV et `/coverage_status` sont écrits par le tracker, indépendamment de l'affichage.
- **Résultat-leçon** : ne pas se fier à l'affichage RViz pour juger d'un run. Les mesures font foi (CSV, `/coverage_status`).

2026-10-09 · Incident 10 : run B bloqué, non résolu

- **Contexte** : run B (Nav2) sur le robot réel, non résolu.
  - Tentative 1 (12 h 26) : Nav2 est actif, mais le log affiche `tf_help: Transform data too old`. Les TF du robot sont environ 2,4 s en retard sur l'horloge du PC. RPP et le Behavior Tree déclarent chaque but atteint sans mouvement : 14 waypoints « réussis » en 12 s, robot immobile.
  - Tentative 2 : tolérances à 1,0 s (`transform_tolerance` pour RPP et les costmaps locale et globale, `transform_timeout` pour le behavior_server). `bt_navigator` reste inactif, car l'activation ne se termine pas. AMCL et la costmap locale signalent « timestamp earlier than all the data in the transform cache ».
- **Options envisagées** : pistes listées dans la séance, non tranchées.
  - Écart d'horloge réel (lire la sortie « écart … ms » de `robot_check.sh`).
  - Latence Wi-Fi des TF.

    - Charge CPU de WSL.
    - Synchronisation propre avec chrony, le PC servant de serveur.
    - Lancer Nav2 sur le Pi du robot.
- **Choix** : non résolu. Aucun run B réel n'est mesurable en l'état.
- **Justification** : les deux tentatives montrent que le statut de Nav2 ne prouve pas le mouvement du robot. Le blocage est lié à la synchronisation des horloges et à la latence des TF.
- **Résultat-leçon** : vérifier le mouvement réel, ne pas se fier au statut de Nav2. Le cœur de la comparaison A/B réelle reste à faire.

2026-10-09 · Constat d'analyse : run RANDOM_2 arrêté à 146,5 s

- **Contexte** : le run RANDOM_2_20261009-121427 était présenté comme valide et complet (72,1 % en 300 s). La relecture des CSV par le chef de projet montre que la distance reste figée à 11,94 m de 146,5 s à 300,5 s, et la couverture à 72,1 %. Pendant ce temps, near_collisions continue de croître (50 → 95) : le LiDAR fonctionne toujours.
- **Options envisagées** : causes possibles, non tranchées faute de rosbag. Arrêt physique (moteurs ou batterie, comme l'incident 6) ou pose figée (TF), comme le run B3 de la simulation.
- **Choix** : le run n'est valide que jusqu'à 146,5 s. Il ne doit pas être présenté comme complet.
- **Justification** : la distance et la couverture sont figées, alors que le LiDAR continue d'envoyer des données. Cela exclut un simple arrêt du capteur, mais pas la cause du blocage.
- **Résultat-leçon** : vérifier la distance parcourue en fonction du temps avant de déclarer un run complet. Enregistrer un rosbag à chaque run réel pour trancher la cause.

2026-10-09 · Constat : free_thresh de la carte zone_reelle

- **Contexte** : la carte `zone_reelle.yaml` a été sauvegardée avec `free_thresh` à 0,25. C'est le même piège que le premier run A du 2026-10-08 : les pixels inconnus sont relus comme libres.
- **Options envisagées** : non consignées.
- **Choix** : les runs de la séance sont mesurés avec 634 cellules atteignables. Le calcul donne 634 cellules avec 0,25 contre 620 avec 0,19, soit un écart de 2 %.
- **Justification** : la zone est fermée par les cartons, donc l'écart reste faible. Il faut tout de même le noter pour l'interprétation des résultats.
- **Résultat-leçon** : vérifier `free_thresh` à chaque sauvegarde de carte, comme le contrôle de cohérence du premier jour.

2026-10-09 · Revue des modifications de la séance robot

- **Contexte** : relecture des modifications faites pendant la séance robot du jour : scripts de vérification et d'arrêt, executor, configuration Nav2, carte `zone_reelle`.
- **Options envisagées** : non consignées.
- **Choix** : deux bloquants trouvés et corrigés.
    - (a) `robot_check.sh` tuait son propre shell distant : `pkill -f motif` s'exécutait dans une commande SSH dont la ligne contenait le motif. Reproduit. Correction : motifs à crochets (`[t]urtlebot3_ros`).
    - (b) `stop_robot.sh` pouvait bloquer indéfiniment, car `ros2 topic pub` attend un abonné. Correction : publications bornées dans le temps ; le script rend la main en 5,4 s, même sans robot.
    - Autres modifications :
        - le `transform_timeout` modifié pendant la séance était placé dans une section ignorée par Humble (`recoveries_server`). Il est déplacé dans la section `behavior_server`.

        - les tolérances à 1,0 s sont un contournement de l'écart d'horloge, pas une correction.
        - les valeurs propres au robot n°1 sont rendues surchargeables.
        - la carte `zone_reelle` est passée à `free_thresh` 0,19.
- **Justification** : (a) évite un double bringup (incident 2). (b) un script d'arrêt doit rendre la main même sans robot.
- **Résultat-leçon** : les pièges déjà rencontrés (`pkill -f`) reviennent quand on écrit vite sous pression. La relecture les rattrape ; elle doit précéder chaque commit de script utilisé sur le terrain.

2026-10-09 · Phase 0 : débloquer le run B sans robot

- **Contexte** : le run B sur le robot réel reste bloqué (incident 10). Les symptômes sont dominés par le temps : l'horloge du robot a été remise à l'heure en cours de séance, et les mémoires de transformations des nœuds du PC gardent des données « du futur ». Le Wi-Fi joue un rôle secondaire. Hypothèse à vérifier sur place.
- **Options envisagées** : latence Wi-Fi des TF, charge CPU de WSL, dérive d'horloge (diagnostic principal retenu). Pistes de l'incident 10 : chrony, Nav2 sur le Pi du robot.
- **Choix** : trois outils avant la prochaine séance.
    - (a) détecteur de progression anormale dans l'executor (module pur `progress_check`). Un waypoint est suspect si la distance odométrique est inférieure à 30 % de la distance attendue ET s'il est validé en moins de 2 s. Alerte au 2e suspect consécutif : but annulé, code de sortie 1.
    - (b) mode `localization:=static` pour B : Nav2 sans AMCL, `map → odom` fixe.
    - (c) script `diag_b.sh` : horloge, âge des messages, fréquences, états Nav2, TF, charge CPU. Option `--sim` .
- **Justification** : le statut de Nav2 ne prouve pas le mouvement du robot. Le détecteur mesure le mouvement. Le mode static retire AMCL des causes possibles. Le diagnostic rend les hypothèses vérifiables.
- **Résultat-leçon** : outils livrés, pas encore validés sur le robot. Le diagnostic reste à faire sur place.

2026-10-09 · Phase 0 : itérations en simulation

- **Contexte** : premier essai en simulation. Le mode `static` donne 7 à 9 % en 150 s, contre 15,4 % avec AMCL. Le détecteur arrête le run dans 2 essais sur 2.
- **Options envisagées** : non consignées.
- **Choix** : l'analyse des logs montre que Nav2 saute les waypoints après un échec (« Failed to process waypoint »), ce qui fait aussi avancer le numéro de waypoint. Corrections :
    - critère de temps ajouté au détecteur. Un faux succès réel dure 0,85 à 0,90 s par waypoint dans les logs de la séance ; les échecs Nav2 en simulation durent 22 s et plus. Ajouter une condition ne peut que réduire les fausses alertes.
    - en mode static, Nav2 est chargé dans un seul conteneur. Avec un processus par nœud, la charge CPU est plus élevée et 1 démarrage sur 4 reste bloqué.
- **Justification** : le numéro de waypoint seul ne suffit pas à juger la progression ; la durée par waypoint sépare un vrai déplacement d'un saut.
- **Résultat-leçon** : static 14,6 à 14,8 % en 150 s (2 runs), contre 15,4 % avec AMCL. Aucune fausse alerte, démarrage au premier essai. Alarme vérifiée : odométrie immobile, critère de temps désactivé, arrêt au 2e waypoint, code 1. 101 tests passent. Limite : un faux succès instantané comme sur le robot ne se reproduit pas en simulation, Nav2 y valide lentement.

2026-10-09 · Incident : démarrage Nav2 bloqué en mode amcl

- **Contexte** : en mode `amcl`, 1 démarrage sur 2 est bloqué. Le conteneur du bringup n'a pas chargé `bt_navigator` (« failed to send response to .../load_node (timeout) »). L'executor attendait indéfiniment.
- **Options envisagées** : non consignées.
- **Choix** : délai d'attente de Nav2 ajouté à l'executor (`nav2_timeout`, 120 s). Au dépassement, sortie avec code 2.
- **Justification** : un processus qui attend sans limite bloque la séance sans message d'erreur exploitable.
- **Résultat-leçon** : un démarrage bloqué doit échouer avec un code de sortie explicite, pas attendre indéfiniment.

2026-10-09 · Incident : port 11345 occupé par Zwift (Gazebo)

- **Contexte** : depuis le passage de WSL en réseau mirrored, Zwift (Windows) occupe le port 11345 de Gazebo. Erreur : « bind: Address already in use ».
- **Options envisagées** : non consignées.
- **Choix** : contournement par `GAZEBO_MASTER_URI=http://localhost:11350`.
- **Justification** : le contournement n'exige pas de fermer Zwift.
- **Résultat-leçon** : en réseau mirrored, les ports Windows et WSL sont partagés. Vérifier les ports occupés avant de lancer Gazebo.

2026-10-09 · Incident : diag_b.sh ne voyait pas la simulation

- **Contexte** : `diag_b.sh` forçait `ROS_LOCALHOST_ONLY=0`, donc il ne voyait pas la simulation. Il lançait aussi trop de commandes `ros2` en parallèle : au-delà d'environ 4, erreur « context is not valid ».
- **Options envisagées** : non consignées.
- **Choix** : option `--sim` ajoutée au script. Au plus 3 commandes `ros2` en parallèle.
- **Justification** : le diagnostic doit utiliser la même configuration réseau ROS que la cible.
- **Résultat-leçon** : un script de diagnostic respecte la configuration réseau ROS de la cible et limite sa concurrence.

2026-10-12 · Impasse : Nav2 sur le robot réel (procédure de déblocage)

- **Contexte** : pendant la séance, la stratégie B lancée par Nav2 sur le robot réel ne produisait aucun déplacement mesurable. La procédure `JOUR_ROBOT.md` section 12 a été automatisée par `scripts/debloque_b.sh` (journaux dans `results/robot_reel/debloque_*`), avec le contrôle de mouvement `scripts/goal_test.py`.
- **Options envisagées** : non consignées.
- **Choix** : diagnostic point par point, un facteur à la fois, avec journalisation de chaque essai.
    1. Horloge : écart ramené de 462 ms à 30-60 ms par une correction relative calculée par le robot dans `robot_check.sh`.
    2. Réseau : âge de `/scan` et `/odom` d'environ 0,26 s ; Nav2 actif (7/7 nœuds).
    3. AMCL : `Transform data too old when converting from map to odom`, avec une transformée map → odom vieille de 27 s. Nav2 déclare le but atteint sans que le robot bouge : faux succès, détecté par la mesure du déplacement réel sur `/odom` (`goal_test.py`).
    4. Mode `static` : Nav2 fait pivoter le robot de 109° à 174° sans jamais avancer, avec `Failed to make progress`. Le progress checker est passé à 0,2 m en 30 s, puis le contrôleur RPP a été testé sans détection de collision : même résultat. Cause non trouvée.

5. Un lancement du suiveur a échoué une fois : `map_server: failed to send response to /map_server/change_state (timeout)`, soit un service DDS perdu de façon intermittente. Relancer a suffi.

- **Justification** : le statut de Nav2 ne prouve pas le mouvement du robot. Seule la mesure sur `/odom` le démontre.
- **Résultat-leçon** :
  - Le statut d'un logiciel ne prouve pas le mouvement : mesurer le déplacement réel.
  - La navigation à travers le Wi-Fi (horloges, latences, services DDS) est un problème en soi, distinct de l'algorithme.
  - Simplifier l'exécutant a débloqué la situation (voir l'entrée suivante).

2026-10-12 · Décision : exécutant simplifié `coverage_follower` pour B sur le robot réel

- **Contexte** : après l'impasse Nav2 sur le robot réel (entrée précédente), la stratégie B devait tout de même être mesurée sur le robot.
- **Options envisagées** : non consignées.
- **Choix** : sur le robot réel, B utilise la même planification boustrophédon que la simulation (`coverage_planner`, module `planning.py`), mais l'exécutant `coverage_follower` remplace Nav2. Principe : tourner puis avancer vers chaque waypoint ; pose lue par TF `map → base_footprint` ; pause si un obstacle est à moins de 0,15 m devant ; waypoint sauté après 40 s sans atteinte. Fichiers : `aspi_bot/follow_logic.py`, `aspi_bot/coverage_follower_node.py`, `launch/aspi_real_follow.launch.py`, `test/test_follow_logic.py`. En simulation, B reste exécuté par Nav2 : c'est la comparaison officielle.
- **Justification** : l'impasse Nav2 sur le robot réel. Un exécutant plus simple, dont chaque action est vérifiable, a débloqué la situation.
- **Résultat-leçon** :
  - Résultat : 2 runs B complets sur le robot réel.
  - Présentation honnête : la planification est identique à la simulation, l'exécution ne l'est pas. La comparaison sur robot réel n'oppose donc pas les mêmes exécutants que la comparaison en simulation.
  - Leçon : quand une pile logicielle lourde bloque, un exécutant plus simple, dont le comportement se mesure, peut suffire à produire une donnée.

2026-10-12 · Résultats réels : stratégie A contre stratégie B sur le robot

- **Contexte** : mesures réelles du 12/10 dans `results/robot_reel/`, avec `localization:=static`, la même carte `zone_reelle` (620 cellules atteignables) et la même croix de départ pour A et B.
- **Options envisagées** : non consignées.
- **Choix** : résultats retenus tels quels ; les runs A du 9/10 (avec AMCL) restent des données d'illustration, non mélangées à ceux-ci. Décision de l'étudiant : robot réel terminé, données conservées telles quelles.
  - A seed 1 (`RANDOM_1_20261012-105211`) : 5,2 % à 30 s, 27,9 % à 60 s, 36,1 % à 120 s, 73,1 % à 300 s. Distance 19,7 m, 21 near_collisions.
  - A seed 3 (`RANDOM_3_20261012-105805`) : 2,6 % à 30 s, 23,2 % à 60 s, 30,3 % à 120 s, 74,8 % à 300 s. Distance 20,9 m, 19 near_collisions.
  - B n°1 (`BOUSTROPHEDON_1_20261012-103146`) : 24,4 % à 30 s, 42,7 % à 60 s, 79,4 % à 120 s, 98,4 % à 300 s ; trajet fini vers 147 s. Distance 9,8 m, 8 near_collisions.
  - B n°2 (`BOUSTROPHEDON_2_20261012-110402`) : 11,9 % à 30 s, 29,7 % à 60 s, 74,5 % à 120 s, 96,5 % à 170 s ; trajet fini vers 157 s, puis arrêt manuel. Distance 9,8 m, 2 near_collisions. Couverture

figée après la fin du trajet.

- **Justification** : mesure dans les mêmes conditions pour A et B. Les courbes réelles se traduisent par un tableau et une courbe `couverture vs temps` (`scripts/plot_results.py` sur `results/robot_reel/`), en signalant que les runs B s'arrêtent avant 300 s.
- **Résultat-leçon** :
  - À 120 s, B atteint environ 75-79 % contre environ 30-36 % pour A.
  - B parcourt deux fois moins de distance et provoque moins de near_collisions.
  - Ces chiffres sont approximatifs : la localisation par odométrie seule dérive (voir l'entrée « Limites »). Deux runs par stratégie ne permettent aucune conclusion statistique.

2026-10-12 · Incident : run B de 10 h 45 arrêté à 161 s

- **Contexte** : run `BOUSTROPHEDON_2_20261012-104557`, arrêté à 161 s. Couverture à 46,8 %, distance 4,6 m, 6 near_collisions. Les waypoints 6 et 7 ont été sautés après 40 s chacun.
- **Options envisagées** : non consignées.
- **Choix** : le run est conservé dans `results/robot_reel/` avec le statut « incident », sans être substitué à un autre. Les notes de séance ne donnent pas d'autre décision.
- **Justification** : cause probable, selon les notes : avec le décalage entre la carte et le réel, un waypoint « libre » sur la carte se trouvait contre un vrai carton. Le suiveur ne pouvait donc pas l'atteindre.
- **Résultat-leçon** :
  - Une cellule libre sur la carte n'est pas forcément libre dans la pièce : un décalage de repère se traduit en blocage.
  - Le délai de 40 s par waypoint limite la perte de temps, sans la supprimer.
  - Ce run montre la limite de la localisation sans LiDAR, traitée dans l'entrée suivante.

2026-10-12 · Limites de la localisation sans LiDAR (odométrie seule, mode static)

- **Contexte** : en mode `localization:=static`, AMCL est retiré : la transformée map → odom est une identité figée. Les conséquences doivent être présentées au jury.
- **Options envisagées** : non consignées.
- **Choix** : limites documentées et expliquées, sans correction dans le code pour cette séance.
  1. Décalage LiDAR / carte visible dans RViz : les murs ne se superposent pas au fond de carte. La transformée figée suppose que le robot est exactement sur la croix et dans l'orientation de la cartographie. Une erreur de cap de 3° donne environ 5 cm d'écart à 1 m. La dérive d'odométrie s'accumule à chaque rotation sur place (le run A tourne environ 20 fois), donc l'écart grandit pendant le run. AMCL, qui recale le LiDAR sur la carte, a été écarté pendant la séance parce qu'il figeait map → odom avec Nav2. Conséquence : le pourcentage couvert est mesuré dans un repère qui dérive. Les chiffres sont approximatifs, surtout pour A.
  2. B s'arrête avant les bords. La marge du planner (robot_radius 0,105 m + safety_margin 0,05 m = 15,5 cm) et la tolérance d'arrivée (8 cm) laissent le robot à environ 20 cm des murs. Le pourcentage est calculé sur la même zone atteignable pour A et B : la bande le long des murs n'est comptée ni nettoyée par aucun des deux. Perspective : une passe finale de suivi de mur.
  3. B se bloque au début : avec le décalage carte/réel, un waypoint « libre » peut se trouver contre un vrai carton. Le suiveur fait alors une pause (obstacle à 0,15 m devant). Lors du run de 10 h 45, deux waypoints inaccessibles ont été sautés après 40 s chacun.
  4. Peu de runs (2 par stratégie), une seule pièce, vitesse réduite à 0,10 m/s pour la sécurité.
- **Justification** : ces limites viennent de l'absence de recalage par LiDAR et de l'odométrie seule, pas d'un défaut de l'algorithme de planification.

- **Résultat-leçon** : sans LiDAR de recalage, le repère du robot dérive et la mesure de couverture s'éloigne de la réalité. Présenter ces limites, plutôt que de les masquer, fait partie de l'évaluation. Perspectives : recalage LiDAR (AMCL ou équivalent) et suivi de mur.

2026-10-12 · Décision : jumeau numérique de la zone réelle pour captures et comparaison

- **Contexte** : le robot réel est terminé. Il faut des captures Gazebo et RViz pour les slides, et une comparaison simulation / réalité sur la même géométrie.
- **Options envisagées** : (a) captures sur `turtlebot3_world`, le monde du protocole de simulation ; (b) jumeau numérique de la zone réelle.
- **Choix** : option (b), demandée par l'étudiant. Le monde Gazebo `worlds/zone_reelle.world` est généré à partir de la carte réelle par `scripts/make_world_from_map.py` : fusion des cellules murs en 96 boîtes de 0,25 m de haut, couverture exacte des 486 cellules murs vérifiée, zone fermée. Un argument `world_file` est ajouté à `launch/sim.launch.py`, vide par défaut : le protocole de simulation reste inchangé.
- **Justification** : les runs du jumeau utilisent les mêmes launch et réglages que le robot réel (`use_sim_time:=true`, 0,10 m/s, mode `static`, 300 s). La zone atteignable compte 620 cellules, comme en réel.
- **Résultat-leçon** : le jumeau permet de comparer simulation et réalité sur la même géométrie. Le protocole de `turtlebot3_world` n'est pas modifié.

2026-10-12 · Résultats : jumeau numérique comparé au robot réel (1 run par cas)

- **Contexte** : un run par configuration sur le jumeau, 300 s. Les CSV sont dans `results/jumeau/`.
- **Options envisagées** : voir l'entrée « Décision : jumeau numérique » ci-dessus.
- **Choix** : résultats consignés tels quels, à comparer avec les runs réels du 12/10.
  - A (RANDOM, seed 1) : jumeau 82,7 % en 300 s, 19,3 m, 28 near_collisions. Réel (seeds 1 et 3) : 73,1 % et 74,8 %, 19,7 m et 20,9 m.
  - B suiveur : jumeau 93,1 %, 9,7 m, 20 near_collisions ; trajet fini vers 159 s ; premier waypoint sauté après 40 s. Réel : 98,4 % et 96,5 %, 9,8 m.
  - B Nav2 : jumeau 67,7 %, 7,6 m, 48 near_collisions en fin de run (le CSV donne 48, la note de séance indiquait 46). Couverture figée dès environ 107 s ; near_collisions passe à 5 vers 125 s, puis augmente.
- **Justification** : même géométrie, mêmes réglages : la comparaison est plus directe qu'entre simulation et réel avec des mondes différents. Un seul run par cas : aucune conclusion statistique.
- **Résultat-leçon** : pour A et B suiveur, le jumeau donne des ordres de grandeur proches du réel. B Nav2 se bloque aussi en simulation (entrée suivante).

2026-10-12 · Constat : Nav2 bloque aussi en simulation sur la zone étroite

- **Contexte** : sur le jumeau, la stratégie B exécutée par Nav2 se bloque vers 125 s contre le mur diagonal près du départ. Les comportements de récupération (recoveries) échouent.
- **Options envisagées** : non consignées.
- **Choix** : aucune correction de Nav2 n'est faite ; le constat est consigné tel quel.
- **Justification** : le blocage se produit en simulation, sans Wi-Fi ni horloge réelle. L'impasse du robot réel n'est donc pas seulement due au réseau ou à l'horloge. Hypothèse à confirmer : un seul run.
- **Résultat-leçon** : Nav2 peut bloquer dans cette zone étroite, même sans les contraintes du réel. Il faudrait plusieurs runs avant de conclure sur la cause.

2026-10-12 · Incident : captures RViz prises derrière le terminal

- **Contexte** : le premier essai de captures par copie de l'écran donnait une fenêtre RViz restée derrière le terminal. Windows refuse de passer une fenêtre au premier plan de force. Les captures de cet essai ont été supprimées.
- **Options envisagées** : copie de l'écran (essai, échoué) ; autres options non consignées.
- **Choix** : capture du contenu de chaque fenêtre par la fonction Windows `PrintWindow`.
- **Justification** : `PrintWindow` capture le contenu d'une fenêtre même si elle est recouverte. La capture ne dépend donc plus de l'ordre des fenêtres.
- **Résultat-leçon** : vérifier une capture avant d'en produire en série.

2026-10-12 · Résultat : 18 captures pour les slides (`results/figures/`)

- **Contexte** : captures Gazebo et RViz pour les trois scénarios du jumeau.
- **Options envisagées** : non consignées.
- **Choix** : 18 captures dans `results/figures/`, nommées `jumeau_<scénario>_<fenêtre>_<instant>.png`. Pour chaque scénario : A (`t030`, `t150`, `t300`), B suiveur (`t030`, `t090`, `fin_trajet`), B Nav2 (`t030`, `t150`, `fin_trajet`), en Gazebo et en RViz.
- **Justification** : ces captures servent de matière aux slides de soutenance et à la vidéo de démonstration.
- **Résultat-leçon** : une capture vérifiée d'abord évite de refaire toute une série.

2026-10-12 · Rangement : CSV de démonstration hors protocole (`results/invalides/`)

- **Contexte** : un run A de démonstration sur `turtlebot3_world` a été fait avant le passage au jumeau : `demo_RANDOM_1_20261012-112040.csv`.
- **Options envisagées** : non consignées.
- **Choix** : fichier rangé dans `results/invalides/`, avec les autres essais hors protocole.
- **Justification** : éviter de mélanger un run de démonstration avec les données du protocole. Les runs du jumeau sont dans `results/jumeau/`.
- **Résultat-leçon** : séparer dès la création les données de démonstration et d'essai des données de protocole.

2026-10-12 · Plan de soutenance final

- **Contexte** : intégrer au plan les résultats réels du 12/10, le jumeau numérique et l'impasse Nav2 sur le robot.
- **Options envisagées** : ajouter des slides au plan existant (≈ 15 min) ou resserrer pour tenir en 10 min.
- **Choix** : 13 slides pour 9 min 30, dont « Robot réel », « Impasse Nav2 sur le robot réel », « Jumeau numérique » et « Limites ». Huit questions du jury ajoutées, dont « pourquoi pas Nav2 sur le robot ? ». PERSPECTIVES : P9 (essai réel) marqué réalisé, ajout de P10 (recalage LiDAR) et P11 (suivi de mur).
- **Justification** : la durée imposée est de 10 min ; les résultats réels sont la meilleure preuve de la démarche.
- **Résultat-leçon** : la relecture a trouvé 5 erreurs, dont 2 chiffres faux et 2 surestimations (« le suiveur a mieux tenu », « même type de blocage »). Toujours recalculer les chiffres depuis les CSV et formuler un résultat sur 1 run comme une hypothèse.

2026-10-12 · Diaporama, fiche du binôme et marge aux murs

- **Contexte** : préparer le diaporama final et une fiche complète pour le coéquipier absent pendant le projet.
- **Options envisagées** : reprendre le plan tel quel ou relire chaque chiffre contre le code avant de le mettre en slide.
- **Choix** : diaporama de 14 slides (titre + 13 du plan), notes d'orateur avec la répartition (binôme : 1, 2, 3, 6, 7, 13, 14 ; Maxime : 4, 5, 8 à 12). Fiche `docs/FICHE_BINOME.md` relue avant commit.
- **Justification** : le jury interroge chaque membre ; le coéquipier doit pouvoir expliquer méthode, chiffres et limites.
- **Résultat-leçon** : la marge d'environ 20 cm aux murs était expliquée par « 15,5 cm d'érosion + 8 cm de tolérance ». Le code (`erosion_radius_cells`) arrondit à la cellule supérieure : 0,155 m deviennent 4 cellules de 5 cm, soit 0,20 m. RESULTATS et PLAN_SOUTENANCE sont corrigés. La relecture a aussi montré qu'un run de démo écrit dans `results/` et fausserait les moyennes si l'on relançait `plot_results.py` : la fiche dirige les démos vers `/tmp/demo`. Leçon : relire le code plutôt que le document quand une explication paraît bricolée.

2026-10-12 · Diaporama simplifié et schémas de principe

- **Contexte** : le diaporama était trop dense pour être défendu slide par slide ; l'enseignant attend des schémas de principe où les topics relient les nœuds jusqu'au robot.
- **Options envisagées** : garder un schéma système unique très complet, ou un schéma simple par stratégie.
- **Choix** : un schéma de principe par stratégie (rectangle = nœud, pilule = topic, flèche = sens des messages), une slide de démo, des emplacements pour les photos et vidéos du robot réel, et moins de chiffres par slide (le détail passe dans les notes d'orateur). 15 slides, aucun code ni chemin de fichier.
- **Justification** : chaque membre doit pouvoir expliquer chaque élément affiché ; un schéma lisible vaut mieux qu'un schéma exhaustif.
- **Résultat-leçon** : la fiche du binôme suit la nouvelle numérotation. Leçon : n'afficher que ce qu'on sait expliquer, garder le reste pour les questions.

2026-10-12 · Fil conducteur du diaporama et livrables en PDF

- **Contexte** : retours sur le diaporama : pas de fil conducteur clair, graphiques surchargés, impasse Nav2 et sa solution mal expliquées, A « meilleur » dans Gazebo incompris.
- **Options envisagées** : retoucher slide par slide, ou reconstruire autour d'une question unique.
- **Choix** : une question (« planifier aide-t-il à mieux aspirer ? »), un sommaire en 4 parties, trois essais racontés comme une enquête (simulation : le plan est bon mais Nav2 perd du temps ; robot réel : Nav2 bloque, un suiveur simple débloque ; jumeau : vérification). Figures épurées générées par `scripts/plot_slides.py` (une idée par figure). Détails techniques en annexes. Documents livrés en PDF dans `livrables/`.
- **Justification** : 10 min de présentation ; le jury doit suivre un raisonnement, les détails servent aux 10 min de questions.
- **Résultat-leçon** : la comparaison « robot parfait 76 % contre Nav2 60 % à distance égale » explique à elle seule pourquoi l'écart A/B est faible en simulation. Leçon : une présentation se construit autour d'une question et de sa réponse, pas autour de la chronologie.